



Mini PC Deployment Guide (All-in-One)

This complete step-by-step guide explains how to host and deploy the **Machine_bot** project on a Mini PC (Ubuntu/Debian Linux or Windows) using **Docker Compose** and **Cloudflare Tunnels**.



Table of Contents

1. [Prerequisites](#)
 2. [Step 1: Install Required Tools](#)
 3. [Step 2: Clone the Project Repository](#)
 4. [Step 3: Setup Cloudflare Tunnel \(Public URL\)](#)
 5. [Step 4: Configure Environment Variables](#)
 6. [Step 5: Build & Launch Docker Containers](#)
 7. [Step 6: Configure Auto-Start on Boot \(Linux\)](#)
 8. [Useful Maintenance Commands](#)
-

1. Prerequisites

- **Mini PC** connected to local network & internet.
 - **Telegram Bot Token** from [@BotFather](#).
 - Internet connection for Cloudflare Tunnels (no port forwarding or static IP needed).
-

Step 1: Install Required Tools

For Ubuntu / Debian Linux:

Run the following commands in your terminal:

```
# 1. Update system packages
sudo apt update && sudo apt upgrade -y

# 2. Install Git and curl
sudo apt install -y git curl wget

# 3. Install Docker & Docker Compose
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# 4. Install Cloudflared (Cloudflare Tunnel CLI)
sudo mkdir -p --mode=0755 /etc/apt/keyrings
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee
/etc/apt/keyrings/cloudflare-main.gpg >/dev/null
echo "deb [signed-by=/etc/apt/keyrings/cloudflare-main.gpg]
https://pkg.cloudflare.com/cloudflared jammy main" | sudo tee
/etc/apt/sources.list.d/cloudflared.list
```

```
sudo apt update && sudo apt install -y cloudflared

# Log out and log back in for Docker group permissions to take effect:
newgrp docker
```

For Windows Mini PC:

1. Install **Git for Windows**: <https://git-scm.com>
2. Install **Docker Desktop**: <https://www.docker.com/products/docker-desktop>
3. Download **Cloudflared binary**: <https://github.com/cloudflare/cloudflared/releases>

Step 2: Clone the Project Repository

```
# Clone the repository to your Mini PC
git clone https://github.com/sanaSOK/Machine_bot.git

# Move into project directory
cd Machine_bot
```

Step 3: Setup Cloudflare Tunnel (Public URL)

Cloudflare Tunnel exposes your local Mini PC web server securely to Telegram WebApp & users worldwide without needing a public IP or open router ports.

Option A: Quick Free Tunnel (Temporary / Testing)

Run this command on your Mini PC to generate an instant free URL:

```
cloudflared tunnel --url http://localhost:5555
```

Output example: <https://your-custom-subdomain.trycloudflare.com>
(Keep this URL copied; you will use it in Step 4).

Option B: Persistent Custom Domain Tunnel (Recommended for Production)

If you own a custom domain on Cloudflare:

1. Log in to your Cloudflare Tunnel:

```
cloudflared tunnel login
```

2. Create a named tunnel:

```
cloudflared tunnel create mini-pc-bot
```

3. Route your domain (e.g., `bot.yourdomain.com`):

```
cloudflared tunnel route dns mini-pc-bot bot.yourdomain.com
```

4. Create configuration file `~/.cloudflared/config.yml`:

```
tunnel: <TUNNEL_UUID>
credentials-file: /home/user/.cloudflared/<TUNNEL_UUID>.json

ingress:
  - hostname: bot.yourdomain.com
    service: http://localhost:5555
  - service: http_status:404
```

5. Install as background system service:

```
sudo cloudflared service install
sudo systemctl start cloudflared
```

Step 4: Configure Environment Variables

Edit `.env.docker` inside the project root directory:

```
nano .env.docker
```

Paste and update your environment configuration:

```
# Node / Port Config
PORT=3000

# Databases Config (Inside Docker network, MySQL runs on host port 3307)
DB_HOST=telegram_mysql
DB_PORT=3306
DB_USERNAME=root
DB_PASSWORD=rootpassword
DB_DATABASE=telegram_app
SUPER_ADMIN_DB_NAME=super_admin_attendances_db
```

```
# Cloudflare Public URL (Replace with your actual Cloudflare Tunnel URL from Step 3)
FRONTEND_URL=https://your-custom-subdomain.trycloudflare.com

# Telegram Bot Credentials
TELEGRAM_BOT_TOKEN=7755106979:AAH...your_bot_token_here...
```

Step 5: Build & Launch Docker Containers

Run Docker Compose to build and start MySQL, Backend, and Frontend containers:

```
# Build and start all services in detached mode
docker compose up -d --build
```

Verify Running Services:

```
docker compose ps
```

You should see 3 active containers:

- `telegram_mysql` (Port 3307:3306)
- `telegram_backend` (Port 3000:3000)
- `telegram_frontend` (Ports 3333:80, 5555:80)

View Live Logs:

```
docker compose logs -f
```

Step 6: Configure Auto-Start on Boot (Linux)

Ensure Docker containers start automatically whenever the Mini PC restarts or experiences a power outage.

1. Enable Docker Service on Boot:

```
sudo systemctl enable docker
```

2. Verify `restart: always` Policy:

The `docker-compose.yml` file is pre-configured with `restart: always` for all services:

- `telegram_mysql`

- `telegram_backend`
- `telegram_frontend`

If the Mini PC reboots, Docker automatically restarts all containers without manual command execution.

Useful Maintenance Commands

Updating Code from GitHub

When you push new changes to GitHub and want to update your Mini PC:

```
cd Machine_bot
git pull origin main
docker compose up -d --build
```

Checking Logs

- **All logs:** `docker compose logs -f`
- **Backend logs:** `docker compose logs -f backend`
- **Frontend logs:** `docker compose logs -f frontend`
- **MySQL logs:** `docker compose logs -f mysql`

Restarting & Stopping

- **Restart containers:** `docker compose restart`
- **Stop containers:** `docker compose stop`
- **Stop & Remove containers:** `docker compose down`

Backing Up Database

To export a full SQL dump from the running Mini PC database:

```
docker exec -t telegram_mysql mysqldump -u root -prootpassword telegram_app >
telegram_app_backup.sql
docker exec -t telegram_mysql mysqldump -u root -prootpassword
super_admin_attendances_db > super_admin_backup.sql
```

Created for Machine_bot Mini PC Deployment.